Injection de dépendances



Objectifs

- Définitions
  - Inversion de contrôle
  - Injection de dépendances
- Cas particulier injection de dépendances
- L’injection de dépendances en C# .Net 8

Inversion de contrôle – Définition

- « L’inversion de contrôle (inversion of control, IoC) est un patron d'architecture commun à tous les frameworks (ou cadre de développement et d'exécution). Il fonctionne selon le principe que le **flot d'exécution** d'un logiciel **n'est plus sous le contrôle** direct de **l'application** elle-même mais du framework ou de la couche logicielle sous-jacente. » (Wikipédia)
- WebForms, WinForms, ASP.Net MVC : le code est **appelé** par le **cadriciel**
- De manière plus générale, une interface utilisateur programmée avec des événements : les développeurs ne contrôlent pas le déroulement de l’application, l’utilisateur oui

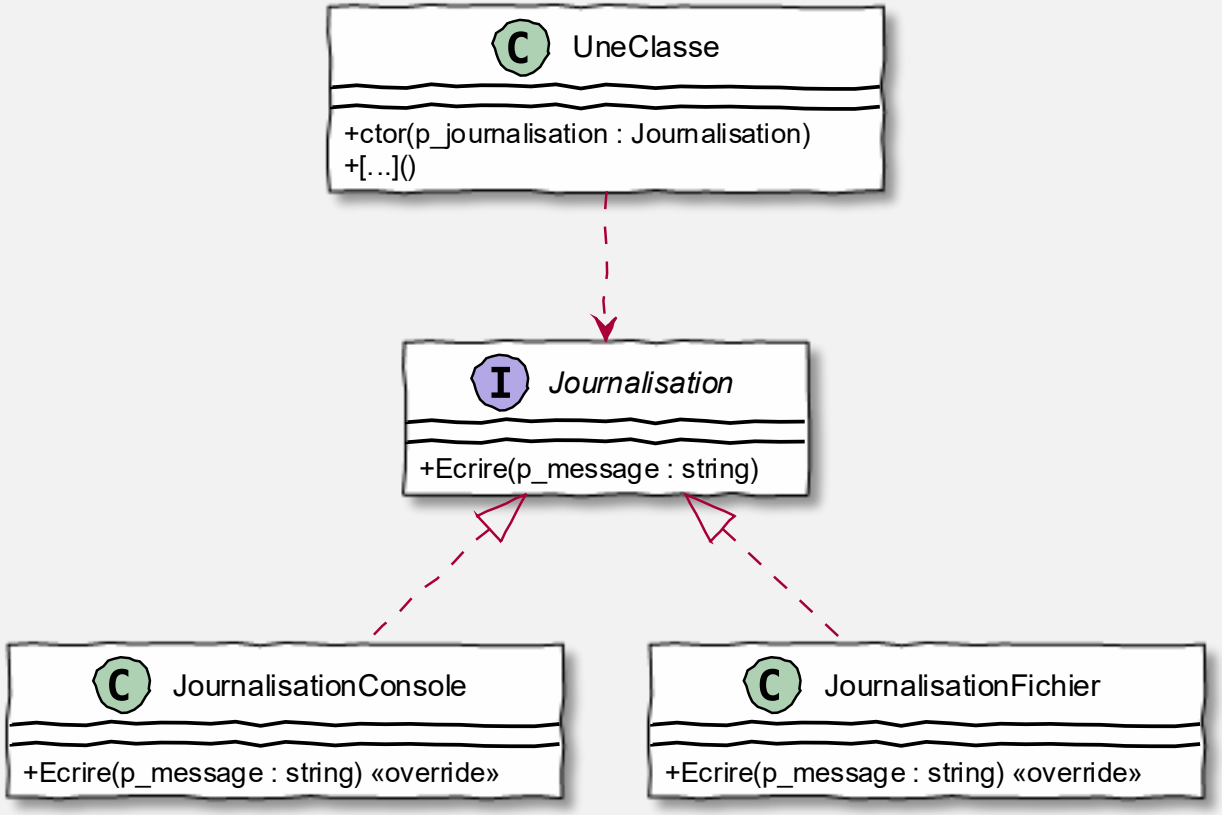
Injection de dépendances – Définition

- « **L'injection de dépendances** (dependency injection en anglais) est un mécanisme qui permet **d'implémenter** le principe de l'inversion **de contrôle.**
  Il consiste à **créer dynamiquement** (**injecter**) les **dépendances** entre les différents objets en **s'appuyant sur une description** (fichier de configuration métadonnées) ou de manière programmatique. Ainsi les **dépendances entre composants** logiciels **ne sont plus exprimées dans le code** de **manière statique mais** déterminées **dynamiquement** à **l'exécution**. » (Wikipédia)
- Dit plus vulgairement, avec l'injection de dépendances, **on ne veut peu d'utilisations de l'instruction « new »** : le programmeur n'a plus le contrôle sur ce qui est créé, c'est la configuration qui le « décide »

Injection de dépendances – Pourquoi

- Le but principal est de limiter les dépendances : vous allez chercher à dépendre d'abstractions et non de classes concrètes (Principe DIP vu en POOII)
- Les applications vont être configurées pour associer des abstractions à des classes concrètes
- Exemple :
  - Vous pouvez avoir une interface « Journalisation »
  - Deux classes concrètes qui l'implantent :
    - Un qui utilise la sortie standard
    - Un qui utilise un fichier texte

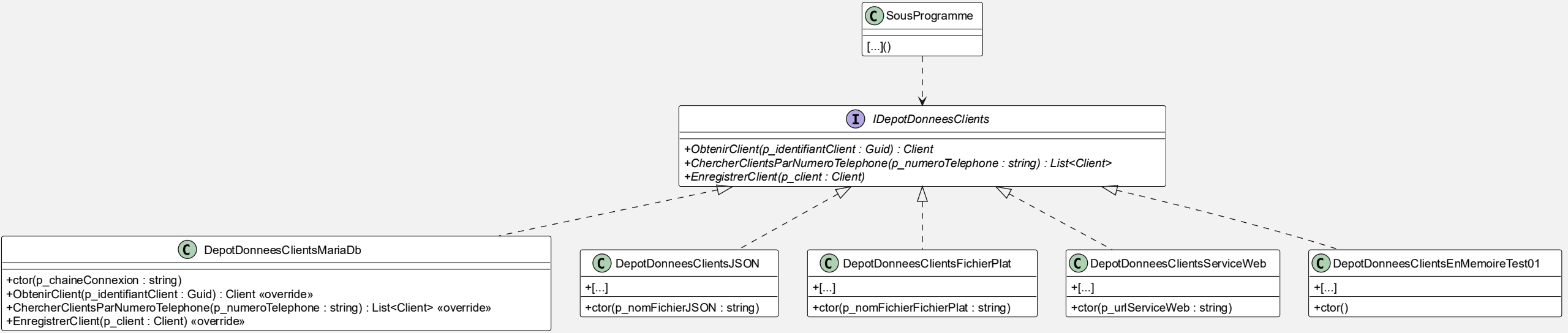
Injection de dépendances – Exemple journalisation



Injection de dépendances – Exemple dépôt de clients

- Vous pouvez avoir une interface « IDepotDonneesClients »
- Cinq classes concrètes qui l'implantent :
  - Une qui utilise le moteur de bases de données « MariaDb »
  - Une qui utilise le format de fichier plat (un client par ligne, champs définis par rapport à son numéro de colonnes)
  - Une qui utilise le format de données JSON
  - Une qui utilise un service web externe
  - Une qui permet de simuler une source de données pour les tests unitaires

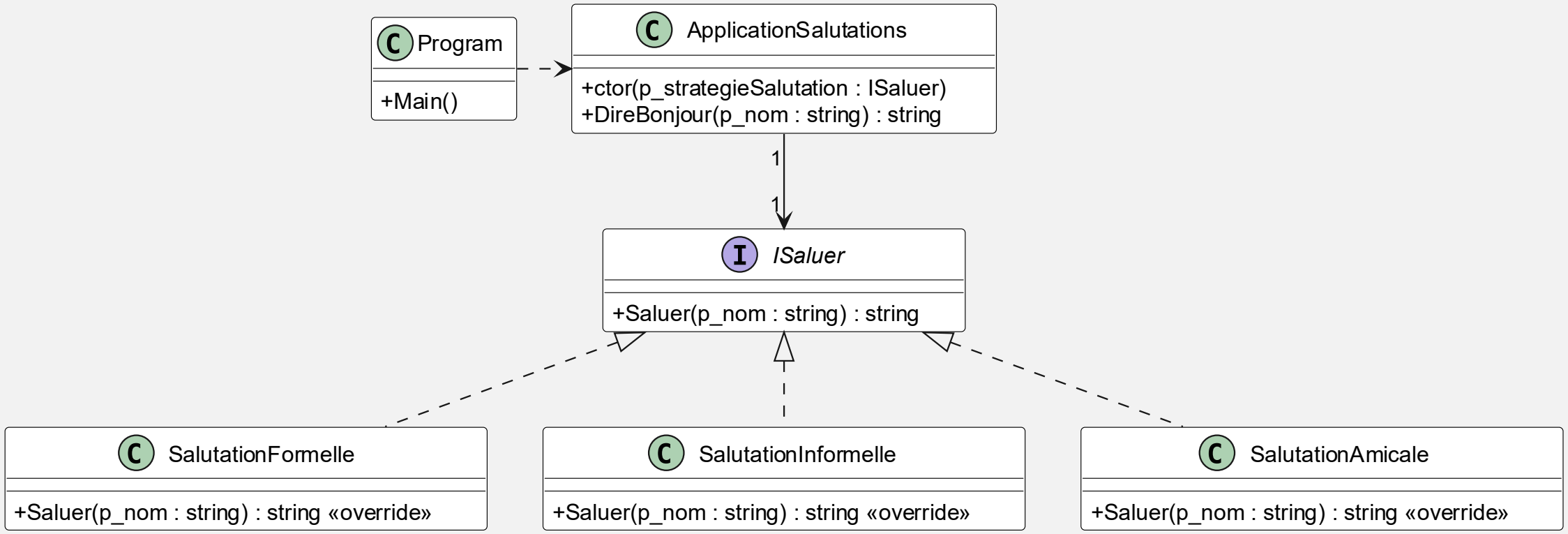
Injection de dépendances – Exemple dépôt de clients



Et C# dans tout cela ?

- En .Net, comme dans beaucoup de cadriciel, il n’y a pas de bibliothèque imposée
- Beaucoup de cadriciel intègrent leur bibliothèque d’injection de dépendances, soit en suggère une
- Dans notre cas, nous allons utiliser la bibliothèque intégrée à .Net 8
- Afin de simplifier le processus de configuration de l’injection de dépendances, nous allons aussi utiliser le package nuget « Microsoft.Extensions.Hosting »

.Net 8 – Exemple – Application de salutations



Code disponible : https://github.com/PiFou86/420-W20-SF/tree/master/Module12_InjectionDependances/ExempleSaluerPrepCours

.Net 8 – Exemple – Application de salutations - Préparation

```
public class ApplicationSalutations
{
	private readonly ISaluer m_saluer;

	public ApplicationSalutations(Isaluer p_strategieSalutation)
	{
		m_saluer = p_strategieSalutation;
	}

	public void Saluer(string p_nom)
	{
		m_saluer.DireBonjour(p_nom);
	}
}
```

.Net 8 – Exemple – Application de salutations - Préparation

```
public interface Isaluer {
  void DireBonjour(string p_nom);
}
```

```
public class SalutationFormelle : Isaluer {
    public void DireBonjour(string p_nom) {
        Console.Out.WriteLine($"Bonjour, Monsieur/Madame {p_nom}. Ravi de faire
    votre connaissance.");
```

```
public class SalutationInformelle : Isaluer {
    public void DireBonjour(string p_nom) {
        Console.Out.WriteLine($"Bonjour, {p_nom}. Ravi de faire ta connaissance.");
```

```
public class SalutationAmicale : Isaluer {
    public void DireBonjour(string p_nom) {
        Console.Out.WriteLine($"Salut, {p_nom}! Content de te voir! Viens-tu prendre
        une bonne chocolatine avec moi ?");
    }
}
```

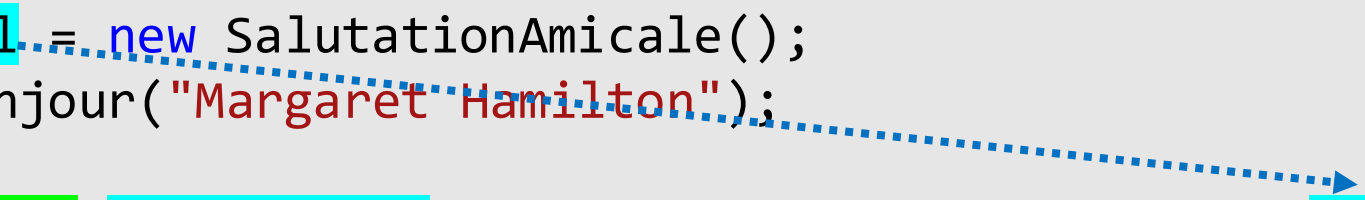
Utilisation classique (Sans injection)

```
ISaluer saluerFormel = new SalutationFormelle();
saluerFormel.DireBonjour("Ada Lovelace");

ISaluer saluerInformel = new SalutationInformelle();
saluerInformel.DireBonjour("Katherine Johnson");

ISaluer saluerAmical = new SalutationAmicale();
saluerAmical.DireBonjour("Margaret Hamilton");

ApplicationSalutations appFormelle = new ApplicationSalutations(saluerAmical);
appFormelle.Saluer("Richard Feynman");
```



Utilisation classique (Sans injection)

```
// Création du conteneur d'injection de dépendances
// Avec sa configuration par défaut (appsettings.json, etc.)
HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

// Enregistrement des services dans le conteneur (i.e. Configuration des dép.)
builder.Services.AddTransient<ISaluer, SalutationFormelle>();
// builder.Services.AddTransient<ISaluer, SalutationInformelle>();
// builder.Services.AddTransient<ISaluer, SalutationAmicale>();

IHost host = builder.Build();

// Résolution de la dépendance et exécution de l'application
ISaluer saluer = host.Services.GetRequiredService<ISaluer>();
saluer.DireBonjour("Dorothy Vaughan");
```

Utilisation classique (Création par IoC et Injection)

```
HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

// Enregistrement des services dans le conteneur
builder.Services.AddTransient<ApplicationSalutations>();

//builder.Services.AddTransient<ISaluer, SalutationFormelle>();
// builder.Services.AddTransient<ISaluer, SalutationInformelle>();
builder.Services.AddTransient<ISaluer, SalutationAmicale>();

IHost host = builder.Build();

// Résolution de la dépendance et exécution de l'application
ApplicationSalutations app = host.Services.GetRequiredService<ApplicationSalutations>();
app.Saluer("Richard Feynman");
```

Cycle de vie

- On distingue généralement trois cycles de vie pour les objets :
  - Transient : une nouvelle instance est créée à chaque résolution de dépendance
  - Scoped : une nouvelle instance est créée par scope (i.e. par requête web, par scope manuel)
  - Singleton : une seule instance est créée pour toute la durée de vie de l'application

- Dans les démonstrations précédentes, le cycle de vie était « Transient », c'est équivalent à un « `new` » à chaque « `GetRequiredService` »

Cycle de vie - Scoped

```
HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

builder.Services.AddScoped<ApplicationSalutations>();

//builder.Services.AddTransient<ISaluer, SalutationFormelle>();
// builder.Services.AddTransient<ISaluer, SalutationInformelle>();
builder.Services.AddScoped<ISaluer, SalutationAmicale>();

IHost host = builder.Build();

// Résolution de la dépendance et exécution de l'application
// Ici on crée un scope, à la fin du scope les objets sont désalloués
using (IServiceScope scope = host.Services.CreateScope())
{
    ApplicationSalutations app =
    scope.ServiceProvider.GetRequiredService<ApplicationSalutations>();
    app.Saluer("Richard Feynman");
}
```

Cycle de vie – Scoped

```
using (IServiceScope scope = host.Services.CreateScope())
{
    ApplicationSalutations app =
    scope.ServiceProvider.GetRequiredService<ApplicationSalutations>();
    app.Saluer("Richard Feynman");
}
```

- Objets de type « `ApplicationSaluation` » et « `ISaluer` »
  - Créés au moment de l'appel du « `GetRequiredService` » :
    - Le moteur liste les constructeurs et choisit le seul disponible qui prends un « `ISaluer` » en paramètres et cherche donc comment le créer. Il trouve l'association « `ISaluer` » avec « `SaluationAmicale` ». Cette classe a un constructeur par défaut. Il crée l'objet avec la durée de vie « `Scoped` ». Il peut maintenant créer un objet de type « `ApplicationSalutations` ».
  - Désalloués à la fin de la portée du scope, donc à la fin du « `using` ».

Injection de dépendances – Aller plus loin

- Documentation .Net 8.0 : https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-8.0

- Code du cours : https://github.com/PiFou86/420-W20-SF/tree/master/Module12_InjectionDependances/ExempleSaluerPrepCours